

数据结构

并查集

家谱

- **核心模型**: 题目要求给出一个父亲的名字, 以及他的儿子们。接下来进行若干次查询, 查询一个人的祖宗
- **思维误区 (Bug)**: 套用dsu模板, 但是模板默认有启发式合并
- **修正逻辑 (Patch)**: 这道题强制要求了父亲与儿子的关系, 所以删掉启发式合并, 把father放在前面son放在后面 (模板默认后面的往前面合并)
- **关键代码**:

DSU (并查集) 里的启发式合并 在并查集中, 启发式合并通常被称为 “按秩合并” (Union by Rank) 或 “按大小合并” (Union by Size)。

它的目的是 防止树退化成链, 从而保证查询速度。

1. 没有启发式合并: 如果每次都固定把 Y 接在 X 下面, 而在极端数据下 (比如 $1 \rightarrow 2, 2 \rightarrow 3, \dots, N-1 \rightarrow N$), 并查集会变成一条长长的“链表”。查找一次祖先需要 $O(N)$ 的时间。
2. 有启发式合并: 我们维护每棵树的 size (节点数) 或 rank (高度)。永远把更小 (或更矮) 的那棵树, 接到更大 (或更高) 的那棵树下面。

```
struct DSU
{
    std::vector<int> fa, sz;
    int count; // 连通块数量

    // 初始化: 传入点数 n (支持 0~n 或 1~n)
    DSU(int n) : fa(n + 1), sz(n + 1, 1), count(n)
    {
        std::iota(fa.begin(), fa.end(), 0); // 0, 1, 2...
    }

    // 查找 + 路径压缩
    int find(int x)
    {
        return x == fa[x] ? x : fa[x] = find(fa[x]);
    }

    // 合并 x 和 y
    // 返回 true 表示合并成功 (原来不连通)
    // 返回 false 表示原来就是连通的
    bool merge(int x, int y)
    {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY)
            return false;
    }
}
```

```

// 启发式合并：把小的树接到大的树下面
// if (sz[rootX] < sz[rootY])
//     std::swap(rootX, rootY);

fa[rootY] = rootX;    // Y 认 X 做爹
sz[rootX] += sz[rootY]; // X 吸收 Y 的人口
count--;             // 连通块少一个
return true;
}

// 判断是否连通
bool same(int x, int y)
{
    return find(x) == find(y);
}

// 获取 x 所在连通块的大小
int size(int x)
{
    return sz[find(x)];
}
};

```

优先队列

P7913 [CSP-S 2021] 廊桥分配

- **核心模型**: 优先队列模拟，读错题
- **思维误区 (Bug)**: wronganser1: 贪心分配航道逻辑错了。wronganser2: 没有释放完他的航道(while写成if)
- 注意重复调用的函数写成lambda 实现小技巧：可以前缀和过去on查询航道占用（贪心序号最小，也就是尽可能地再前面的航道放更多地飞机）
- **修正逻辑 (Patch)**:
- **关键代码**:

```

struct tim
{
    int l, r;
};

void solve()
{
    int n, m1, m2;
    cin >> n >> m1 >> m2;

    vector<tim> gn(m1);
    vector<tim> gw(m2);

```

```

rep(i, 0, m1 - 1) cin >> gn[i].l >> gn[i].r;
rep(i, 0, m2 - 1) cin >> gw[i].l >> gw[i].r;

auto px = [&](vector<tim> &a,vi &pre) -> void
{
    priority_queue<pii, vector<pii>, greater<pii>> man;
    priority_queue<int, vi, greater<>> xian;

    vi cnt(n + 2, 0);
    int idx = 0;
    for (auto it : a)
    {
        while(!man.empty() && it.l > man.top().first)
        {
            xian.push(man.top().second);
            man.pop();
        }

        int now=0;
        if(!xian.empty())
        {
            now=xian.top();
            xian.pop();
        }
        else
        {
            idx++;
            now=idx;
        }
        if(now<=n)
        {
            cnt[now]++;
            man.push({it.r,now});
        }
    }

    for(int i=1;i<=n;i++)
    {
        pre[i]=cnt[i]+pre[i-1];
    }
};

sort(all(gn), [&](tim a, tim b)
    { return a.l < b.l; });
sort(all(gw), [&](tim a, tim b)
    { return a.l < b.l; });

vi pre1(n+2);
vi pre2(n+2);

px(gn,pre1);
px(gw,pre2);

```

```

    ll ans=-1;
    for(int i=0;i<=n;i++)
    {
        ans=max(ans,(ll)pre1[i]+pre2[n-i]);
    }
    cout<<ans<<'\n';
}

```

单调栈

玉蟾宫

- **核心模型**:单调栈求最大矩形面积，找到以每一行为底的高度（直接累加，线性处理）
- **思维误区 (Bug)**:单调栈方向反了，以及单调栈里面留下的最后一个元素是最远阻挠元素，我们的通行范围是阻挠元素往里面一点的那一段
- **修正逻辑 (Patch)**:记得检查数据范围
- **一些思维上的疑惑**:
 - 中间消失的那些元素，一定比栈里留下的元素“高”或者“等高”
 - 计算合法的原因是因为维护的左边和右边由于他的特性一定是匹配的两个
 - 单调栈的特性：找到最近的那一个，因为元素是有序的所以可以保证逻辑是线性的；
 - 这份代码里面的单调栈查找逻辑：对于每个点（这一行）维护他的往左区间，**所以更新在emplace前更新（注意更新时间）**由于单调栈的线性逻辑特性top永远是最近的。
- **关键代码**:

```

void solve()
{
    int n, m;
    cin >> n >> m;
    vector<vi> mp(n, vi(m, 0));

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            char d;
            cin >> d;
            if (d == 'F')
                mp[i][j] = 1;
        }
    }

    vi h(m);
    vi l(m), r(m);
    ll ans = 0;
    for (int i = 0; i < n; i++)
    {

```

```
for (int j = 0; j < m; j++)
{
    if (mp[i][j])
    {
        h[j]++;
    }
    else
        h[j] = 0;
}

stack<int> stk;
for (int j = 0; j < m; j++)
{
    while (!stk.empty() && h[stk.top()] >= h[j])
    {
        stk.pop();
    }

    if (stk.empty())
    {
        l[j] = 0;
    }
    else
    {
        l[j] = stk.top()+1;
    }

    stk.emplace(j);
}

stack<int> stk1;
for (int j = m - 1; j >= 0; j--)
{
    while (!stk1.empty() && h[stk1.top()] >= h[j])
    {
        stk1.pop();
    }

    if (stk1.empty())
    {
        r[j] = m-1;
    }
    else
    {
        r[j] = stk1.top() -1;
    }

    stk1.emplace(j);
}

for (int j = 0; j < m; j++)
    ans = max(ans, (ll)(r[j] - l[j] + 1) * h[j]);
}
```

```
    cout << ans * 3;
}
```

ImbalancedArray

- **核心模型**:推式子模拟计算
- **思维误区 (Bug)**:记得计算贡献的时候, 左闭右闭区间看的是点不是长度! 所以要+1.以及你的操作是左维护一个值右边维护一个值, 记得有一个区间开一个区间闭! 以下提供ac代码的完美的去重逻辑
- **修正逻辑 (Patch)**
 - 要求区间最大值和最小值的差。进行简单的数学划分就能划出来-> $sum = \{min\} + \{max\}$.
 - 对于每个数来说, 计算他能影响到的区间, 也就是他可以给总结果的贡献
 - 根据乘法原理可以算出一个数字他可以贡献的价值为 $nums[i]$, 他贡献的次数是他作为最大值/最小值出现在某个区间的组合种类数-> $ans += (ll) (nums[i] \times (i - le[i] + 1) \times (ri[i] - i + 1));$ (乘法原理)
 - **单调栈就是用来维护这种==最近区间==的**
- **关键代码**:

```
void solve()
{
    int n;
    cin >> n;
    vi nums(n + 1);
    rep(i, 1, n)
    {
        cin >> nums[i];
    }

    vi le(n + 1);
    vi ri(n + 1);
    //-----维护最大值-----//

    stack<int> stk1;
    for (int i = 1; i <= n; i++)
    {
        while (!stk1.empty() && nums[stk1.top()] <= nums[i])//区间闭
        {
            stk1.pop();
        }
        if (stk1.empty())
        {
            le[i] = 1;
        }
        else
        {
            le[i] = stk1.top() + 1;
        }
        stk1.emplace(i);
    }
}
```

```

stack<int> stkr;
for (int i = n; i >= 1; i--)
{
    while (!stkr.empty() && nums[stkr.top()] < nums[i])//区间开
    {
        stkr.pop();
    }
    if (stkr.empty())
    {
        ri[i] = n;
    }
    else
    {
        ri[i] = stkr.top() - 1;
    }
    stkr.emplace(i);
}
ll ans = 0;
for (int i = 1; i <= n; ++i) // 计算贡献
    ans +=(ll) (nums[i] * (i - le[i]+1) * (ri[i] - i+1));
//-----维护最小值-----//

vi l(n + 1);
vi r(n + 1);
stack<int> stl;
for (int i = 1; i <= n; i++)
{
    while (!stl.empty() && nums[stl.top()] >= nums[i])
    {
        stl.pop();
    }
    if (stl.empty())
    {
        l[i] = 1;
    }
    else
    {
        l[i] = stl.top() + 1;
    }
    stl.emplace(i);
}
stack<int> str;
for (int i = n; i >= 1; i--)
{
    while (!str.empty() && nums[str.top()] > nums[i])
    {
        str.pop();
    }
    if (str.empty())
    {
        r[i] = n;
    }
    else
    {

```

```

        r[i] = str.top() - 1;
    }
    str.emplace(i);
}

for (int i = 1; i <= n; ++i) // 计算贡献
    ans -= (ll)(nums[i] * (i - l[i]+1) * (r[i] - i+1));

cout << ans << '\n';
}

```

良好的感觉

- **核心模型**:单调栈维护某个最小值的“生效区间”
- **思维误区 (Bug)**:单调栈的边界判断-悬线法
- **修正逻辑 (Patch)**:左边边界收缩 (+1) 一点, 右边边界收缩 (-1) 一点
- **单调栈逻辑**: 维护最小值堆里面留下大的, 维护最大值堆里面留下小的
- **关键代码**:

```

void solve()
{
    int n;
    cin >> n;
    int d;
    vi qzh(n + 1);
    vi nms(n + 1);
    rep(i, 1, n)
    {
        cin >> d;
        nms[i] = d;
        qzh[i] = qzh[i - 1] + d;
    }

    vi l(n + 1);
    vi r(n + 1);
    stack<int> stl;
    for (int i = 1; i <= n; i++)
    {
        while (!stl.empty() && nms[stl.top()] >= nms[i])
        {
            stl.pop();
        }
        if (stl.empty())
        {
            l[i] = 1;
        }
        else
        {
            l[i] = stl.top() + 1;
        }
    }
}

```

```

        stl.emplace(i);
    }
    stack<int> str;
    for (int i = n; i >= 1; i--)
    {
        while (!str.empty() && nms[str.top()] > nms[i])
        {
            str.pop();
        }
        if (str.empty())
        {
            r[i] = n;
        }
        else
        {
            r[i] = str.top()-1;
        }
        str.emplace(i);
    }
    ll ans = 0;
    for (int i = 1; i <= n; i++)
    {
        ll sumq = qzh[r[i]] - qzh[l[i]-1];
        ll qq = nms[i];
        // cerr<<sumq<<' '<<qq<<' '<<nms[i]<<"sss"<<l[i]<<' '<<r[i]<<'\n';
        ans = max(ans, sumq * qq);
    }
    cout<<ans;
}

```

F. Rae Taylor and Trees (hard version)

- **核心模型**:单调栈维护树状连通块, 难点在连通块连接
- **思维误区 (Bug)**:记录第一直觉为什么错了 (如: 以为是DP其实是贪心 / 读错题)
- **修正逻辑 (Patch)**:
 - 连通块链接处理方法->维护前缀最小值, 链接到前缀最小值头上去
 - 错误逻辑二: 单调栈写错了
- **关键代码**:

```

void solve()
{
    int n;
    cin >> n;
    vi nums(n+1);
    vi pre(n+1);
    pre[0]=n+1;
    rep(i, 1, n)
    {
        cin >> nums[i];
    }
}

```

```

        pre[i]=min(nums[i],pre[i-1]);
    }

    auto cek = [&]() -> bool
    {
        int mn = -1;
        for (int i = n , cnt = 1; i > 1; i--, cnt++)
        {
            mn = max(mn, nums[i]);

            if (mn == cnt)
            {
                cout << "No" << '\n';
                return 0;
            }
        }
        return 1;
    };

    if (!cek())
        return;

    vi fa(n + 1);
    stack<int> stk;
    vi suf;
    for (int i = n; i > 0; i--)
    {
        while (!stk.empty() && nums[stk.top()] < nums[i])
        {
            stk.pop();
        }
        if (stk.empty())
        {
            fa[i] = 0;
            suf.push_back(-1*i);
        }
        else
        {
            fa[i]=stk.top();
        }
        stk.emplace(i);
    }

    vi ans(n+1);
    for(int i=n;i>0;i--)
    {

        if(fa[i]==0&&nums[i]!=n)
        {
            int idx;
            if(upper_bound(all(suf),-1*i)!=suf.end())
            {
                idx=-1*(upper_bound(all(suf),-1*i));
            }
        }
    }

```

```

        else idx=i-1;
        ans[i]=pre[idx];
    }
    else ans[i]=nums[fa[i]];
}

cout << "Yes" << '\n';
for(int i=1;i<=n;i++)
{
    if(nums[i]==n)continue;
    cout<<nums[i]<< ' ' << ans[i]<< '\n';
}

}

```

D. Simons and Beating Peaks

- **核心模型**:非常常见的拼接型找最大值
- **思维误区 (Bug)**:
- **修正逻辑 (Patch)**:
- **关键代码**:

```

void solve()
{
    int n;
    cin >> n;
    vi nums(n);
    int mx = -1;
    int tpmx = 0;

    rep(i, 0, n - 1)
    {
        cin >> nums[i];
    }

    auto ddz = [&]() -> vi
    {
        vi dl(n + 1);
        stack<int> stk;
        for (int i = 0; i < n; i++)
        {
            while (!stk.empty() && nums[stk.top()] < nums[i])
            {
                stk.pop();
            }
            if (stk.empty())
            {
                dl[i] = 0;
            }
        }
    };
}

```

```

        }
        else
        {
            dl[i] = stk.size();
        }
        stk.push(i);
    }
    return dl;
};

auto ddz2 = [&]() -> vi
{
    vi dl(n + 1);
    stack<int> stk;
    for (int i = n - 1; i >= 0; i--)
    {
        while (!stk.empty() && nums[stk.top()] < nums[i])
        {
            stk.pop();
        }
        if (stk.empty())
        {
            dl[i] = 0;
        }
        else
        {
            dl[i] = stk.size();
        }
        stk.push(i);
    }
    return dl;
};

vi left = ddz(), right = ddz2();
int ans = INF;
for (int i = 0; i < n; i++)
{
    auto lf = i - left[i];
    auto rt = n - 1 - i - right[i];
    dbg(left[i], right[i]);
    ans = min(ans, lf + rt);
}
cout << ans << '\n';
}

```

C. Where's My Water?

- ****注意:**** 以下是复杂度 $O(n)$ 的解法，该题数据可以用 n 方
- **核心模型:** 单调栈 $O(n)$ 计算每个出水口能吸收的水（有点类似于玉蟾宫）然后用天才贪心固定住一个点找第二个
- **修正逻辑 (Patch):** 这里只介绍关于单调栈维护的部分：单调栈可以做到 $O(n)$ 去从前往后维护。举个例子：往左找的：我们维护一个单调递减的栈（栈里面留下严格大于的土柱）这里的维护逻辑类似于玉蟾宫悬

线法：这里的wl (wr) 是用类似dp的一个方法维护 维护方法是维护到目前的四方格里面有多少土块被覆盖 最后更新用一个容斥做，（不需要考虑左闭右开...）

```

vi sl(n + 2);
vi wl(n + 2);
vi sum(n + 2);
stack<int> stl;
for (int i = 1; i <= n; i++)
{
    while (!stl.empty() && nums[stl.top()] <= nums[i])
    {
        stl.pop();
    }
    int now = 0;
    if (stl.empty())
    {
        now = 0;
    }
    else
    {
        now = stl.top();
    }
    sl[i] = sl[now] + (i - now) * nums[i];
    wl[i] = i * h - sl[i];
    stl.emplace(i);
}
vi sr(n + 2);
vi wr(n + 2);
stack<int> str;
for (int i = n; i >= 1; i--)
{
    while (!str.empty() && nums[str.top()] <= nums[i])
    {
        str.pop();
    }
    int now = 0;
    if (str.empty())
    {
        now = n + 1;
    }
    else
    {
        now = str.top();
    }
    sr[i] = sr[now] + (now - i) * nums[i];
    wr[i] = (n - i + 1) * h - sr[i];
    str.emplace(i);
}

pii mx_ans = {0, 0};
for (int i = 1; i <= n; i++)
{

```

```

        sum[i] = wl[i] + wr[i] - (h - nums[i]);
        if (mx_ans.first < sum[i])
        {
            mx_ans = {sum[i], i};
        }
    }
}

```

- 关键代码:

```

void solve()
{
    int n, h;
    cin >> n >> h;
    vi nums(n + 2);

    rep(i, 1, n)
    {
        int d;
        cin >> d;
        nums[i] = d;
    }

    vi sl(n + 2);
    vi wl(n + 2);
    vi sum(n + 2);
    stack<int> stl;
    for (int i = 1; i <= n; i++)
    {
        while (!stl.empty() && nums[stl.top()] <= nums[i])
        {
            stl.pop();
        }
        int now = 0;
        if (stl.empty())
        {
            now = 0;
        }
        else
        {
            now = stl.top();
        }
        sl[i] = sl[now] + (i - now) * nums[i];
        wl[i] = i * h - sl[i];
        stl.emplace(i);
    }
    vi sr(n + 2);
    vi wr(n + 2);
    stack<int> str;
    for (int i = n; i >= 1; i--)
    {
        while (!str.empty() && nums[str.top()] <= nums[i])

```

```

        {
            str.pop();
        }
        int now = 0;
        if (str.empty())
        {
            now = n + 1;
        }
        else
        {
            now = str.top();
        }
        sr[i] = sr[now] + (now - i) * nums[i];
        wr[i] = (n - i + 1) * h - sr[i];
        str.emplace(i);
    }

    pii mx_ans = {0, 0};
    for (int i = 1; i <= n; i++)
    {
        sum[i] = wl[i] + wr[i] - (h - nums[i]);
        if (mx_ans.first < sum[i])
        {
            mx_ans = {sum[i], i};
        }
    }

    int ans2 = mx_ans.first;
    int idx = mx_ans.second;
    int curMax = nums[idx];
    int curtp = idx;
    // cerr<<idx;
    for (int j = idx; j >= 1; j--)
    {
        if (curMax < nums[j])
        {
            curtp = j;
            curMax = nums[j];
        }

        ans2 = max(ans2, sum[idx] + sum[j] - sum[curtp]);
    }

    curMax = nums[idx];
    curtp = idx;
    for (int j = idx + 1; j <= n; j++)
    {
        if (curMax < nums[j])
        {
            curtp = j;
            curMax = nums[j];
        }

        ans2 = max(ans2, sum[idx] + sum[j] - sum[curtp]);
    }

```

```

    }

    cout << ans2 << '\n';
}

```

关于单调栈得两种写法

由于学习了悬线法之后脑子变成石头了总是搞反两种写法：以下时讲解

- 倒序遍历

```

vector<int> nextGreaterElementBackward(const vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, -1); // 初始化为 -1
    stack<int> st; // 存下标

    // 从右往左遍历
    for (int i = n - 1; i >= 0; i--) {
        // 1. 维护单调性：把栈顶比当前元素 nums[i] 小的（或相等的）都踢掉
        // 因为 nums[i] 挡在了它们前面，且 nums[i] 更大，所以它们不可能是左边元素的答案
        while (!st.empty() && nums[st.top()] <= nums[i]) {
            st.pop();
        }

        // 2. 记录答案：现在的栈顶就是右边第一个比我大的
        if (!st.empty()) {
            ans[i] = nums[st.top()];
            // 如果题目要求存下标，这里就写 ans[i] = st.top();
        }

        // 3. 入栈：把自己放进去，作为左边元素的备选
        st.push(i);
    }
    return ans;
}

```

- 正序遍历

```

vector<int> nextGreaterElementForward(const vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, -1); // 初始化为 -1
    stack<int> st; // 【必须】存下标，否则不知道答案该填给谁

    // 从左往右遍历
    for (int i = 0; i < n; i++) {
        // 1. 结算：如果当前 nums[i] 比栈顶元素大
        // 说明栈顶那个家伙终于等到它的“右边第一个大数”了
        while (!st.empty() && nums[i] > nums[st.top()]) {

```

```

        int index = st.top(); // 取出等待者的下标
        st.pop();
        ans[index] = nums[i]; // 给这个下标填上答案
    }

    // 2. 入栈: 当前 nums[i] 还没找到比它大的, 入栈等待
    st.push(i);
}
return ans;
}

```

以下是几个**“不能（或很难）互换”**的关键场景：1 涉及到“数据依赖”的 DP 问题 (Backward 胜出) 2. 悬线法 / 柱状图最大矩形 (Forward 的一次遍历优化) 3. 字典序要求 (特殊场景) 4. 环形数组 (Forward 略微好写)

单调队列

切蛋糕

- **核心模型**: 含负数的求最大子段和（长度可变）zon 用单调队列
- **思维误区 (Bug)**: 含负数的求最大子段和/注意！！这个循环要从 $i=0$ 开始因为我们实在做前缀和计算区间长度，我们需要计算进去 $a[1]$
- **修正逻辑 (Patch)**:
- **关键代码**:

```

void solve()
{
    int n, k;
    cin >> n >> k;
    vi nums(n + 1);
    vi qzh(n + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> nums[i];
        qzh[i] = qzh[i - 1] + nums[i];
    }
    deque<int> dq;
    vi da(n + 1);
    ll ans = -LINF;
    for (int i = 0; i <= n; i++)
    {
        while (!dq.empty() && dq.front() + k < i)
        {
            dq.pop_front();
        }
        if (!dq.empty())
        {

```

```

        ans=max(ans,qzh[i]-qzh[dq.front()]);
    }
    while (!dq.empty() && qzh[dq.back()]>=qzh[i])
    {
        dq.pop_back();
    }

    dq.emplace_back(i);

}
cout<<ans<<'\n';
}

```

O - 【典题】DP / 单调队列优化DP ★

- **核心模型**:单调队列, dp
- **思维误区 (Bug)**:“如果一个新人 (新元素), 比你年轻 (下标更大), 还比你强 (数值更大), 那你就是毫无价值的, 可以直接退役 (出队) 了。”
- **修正逻辑 (Patch)**:单调队列能做的在数据范围小的时候能用multiset或优先队列逃课nlogn
- **关键代码**:

```

void solve()
{
    int n, l, r;
    cin >> n >> l >> r;

    vi nums(n + 1);

    for (int i = 0; i <= n; i++)
    {
        cin >> nums[i];
    }
    vi dp(n + 1, -INF);
    deque<int> dq;
    dp[0] = 0;
    ll ans = -INF;

    for (int i = 1; i <= n; i++)
    {
        if (i - r)
        {
            while (!dq.empty() && i - r > dq.front())
            {
                dq.pop_front();
            }
        }
        if (dp[i - 1] > -INF)

```

```

    {
        while (!dq.empty() && dp[dq.back()] < dp[i - 1])
        {
            dq.pop_back();
        }
        dq.push_back(i - 1);
    }
    if(!dq.empty())
        dp[i] = max(dp[i], dp[dq.front()] + nums[i]);
    if (i + r > n)
    {
        ans = max(ans, dp[i]);
    }
}
cout << ans << '\n';
}

```

树状数组

R - 【典题】逆序对 ★

- **核心模型**:离散化, 桶排;
- **思维误区 (Bug)**:读题错误:逆序对要求是 $p[i]>p[j]$ 严格!
- **修正逻辑 (Patch)**:记得离散化。想要快速查到见过的元素有多少, 可以用树状数组维护, 便捷的维护能加起来的桶 (经典离线运用见V - 【典题】静态区间种类数 / HH的项链 ★ ★)
- **关键代码**:

```

void solve()
{
    int n;
    cin >> n;
    Fenwick tr(n + 3);
    ll sum = 0;
    vi nums(n);
    for (int i = 0; i < n; i++)
    {
        cin >> nums[i];
    }
    vi num = nums;
    sort(all(num));
    num.erase(unique(all(num)), num.end());
    auto get_id = [&](int x)
    {
        return lower_bound(all(num), x) - num.begin() + 1;
    };
    //reverse(all(nums));
    for (int i = 0; i < n; i++)

```

```

    {
        sum += tr.range_ask(get_id(nums[i]-1)+1,n+1);
        //cerr<<get_id(nums[i])<<' ';
        tr.add(get_id(nums[i]), 1);
    }
    cout << sum << '\n';
}

```

堆

P1631 序列合并

- **核心模型**:堆排序, 二叉树
- **思维误区 (Bug)**:用一种类似二叉树或者说图论的方式想, 但是发现有冲突! 但是一开始的做法是死板地重复导入左节点和右节点, 会导致冲突 (重复入堆)。简单来说就是: 由于 $x+1$ 和 $y+1$ 是并列的维度, 一定存在菱形结构 (重复)。我们优先导入 $(x, 0)$ 所有数, 这样子保证了不会乱扩展入队。失败原因在用了二叉树的搜索策略去搜索网格图
- **修正逻辑 (Patch)**:
- **关键代码**:

```

int n;
cin >> n;
vi a(n);
vi b(n);
for (int i = 0; i < n; i++)
{
    cin >> a[i];
}
for (int i = 0; i < n; i++)
{
    cin >> b[i];
}
sort(all(a));
sort(all(b));
priority_queue<pair<int, pii>, vector<pair<int, pii>>, greater<>> pq;
int x = 0, y = 0;
// pq.emplace(a[x] + b[y], make_pair(x, y));

for(int i=0;i<n;i++)
{
    pq.emplace(a[i] + b[0], make_pair(i, 0));
}
for (int i = 0; i < n; i++)
{
    // if(x+1<n)
    // pq.emplace(a[x+1]+b[y],make_pair(x+1,y));
    if(y+1<n)

```

```

        pq.emplace(a[x]+b[y+1],make_pair(x,y+1));

        cout<<pq.top().first<<' ';
        pq.pop();
        x=pq.top().second.first;
        y=pq.top().second.second;
    }

```

字典树

- 找最大xor对, 查前缀

最大异或对

- **核心模型**:贪心最高位往下 (前缀差异长度)
- **思维误区 (Bug)**:忘记字典树每一位都需要递归下去
- **修正逻辑 (Patch)**:字典树的两个循环本质是递归所以记得更新指针
- **关键代码**:

```

struct Trie
{
    int tre[MAXN][2];
    int cnt[MAXN];
    int idx = 0;

    void clear()
    {
        for (int i = 0; i <= idx; i++)
        {
            for (int j = 0; j < 2; j++)
                tre[i][j] = 0;
            cnt[i] = 0;
        }
        idx = 0;
    }

    int getnum(char ch)
    {
        // if (ch >= 'A' && ch <= 'Z')
        // {
        //     return ch - 'A';
        // }
        // else if (ch >= 'a' && ch <= 'z')
        // {
        //     return ch - 'a' + 26;
        // }
        // else
        if (ch >= '0' && ch <= '9')
        {
            return ch - '0';
        }
    }
}

```

```

    }
}

void insert(string s)
{
    int ptr1 = 0;
    int bra = 0;
    for (int i = 0; i < s.size(); i++)
    {
        if (!tre[ptr1][getnum(s[i])])
        {
            idx++;
            tre[ptr1][getnum(s[i])] = idx;
        }
        ptr1 = tre[ptr1][getnum(s[i])];
        cnt[ptr1]++;
    }
}

int query(string s)
{
    int ans=0;
    int ptr2 = 0;
    for (int i = 0; i < s.size(); i++)
    {
        if (tre[ptr2][getnum(s[i])^1])
        {
            ptr2=tre[ptr2][getnum(s[i]^1)];
            ans+= (1ll << (s.size() -1 - i));
            // cerr<<(1 << (s.size() - i));
        }
        else
        {
            ptr2=tre[ptr2][getnum(s[i])];
        }
    }
    return ans;
}
} myTrie;

void solve()
{
    int n;
    cin>>n;

    auto tos=[&](int d)->string
    {
        int q=d;
        string s="";
        while(d>0)
        {
            s+=(d%2)+'0';
            d/=2;
        }
    }
}

```

```
        int yy=(31-s.size());
        while(yy--)
        {
            s+='0';
        }
        std::reverse(s.begin(), s.end());
        return s;
    };
    vi nums(n);
    for(int i=0;i<n;i++)
    {
        int d;
        cin>>d;
        //cerr<<tos(d)<<'\n';
        myTrie.insert(tos(d));
        nums[i]=d;
    }
    vi jb(n);
    for(int i=0;i<n;i++)
    {
        jb[i]=myTrie.query(tos(nums[i]));
        // cerr<<jb[i]<<' ';
        cerr<<'\n';
    }
    cout<<*max_element(all(jb));
}
```
